



PERFORMANCE AND DEBUGGING TOOLS

Laura Morgenstern, Developer Technology Engineer @ Nvidia | ISC26



Goals

In this session you will learn how to...

- **Debug** CUDA programs with `cuda-gdb`
 - Approach **correctness checking** with the *Compute Sanitizer* tool suite
 - Use *Nsight* **profiling** tool suite to guide performance optimization of GPU programs
- ➡ Working code
 - ➡ *Actually* working code
 - ➡ Fast(er) code



CORRECTNESS CHECKING

compute-sanitizer

Functional correctness checking suite for GPU

- `compute-sanitizer` is a collection of tools
- `memcheck` (default) tool comparable to [Valgrind's memcheck](#)
- Other tools include
 - `racecheck`: shared memory data access hazard detector
 - `initcheck`: uninitialized device global memory access detector
 - `synccheck`: identify whether a CUDA application is correctly using synchronization primitives
- Focus on `memcheck` in this session, please find examples for other tools in the [Compute Sanitizer Tool Examples](#)

compute-sanitizer

Capabilities of memcheck

- Auto-detect invalid memory accesses to shortcut debugging effort
 - Directly pinpoint source code line, addresses, access size and execution context
- Checking for memory leaks from device allocations  Forgot to call `cudaFree()`?
 - `--leak-check full`
- Filtering and other capabilities; two commonly useful switches:
 - `--log-file output.log`
 - Separates (potentially verbose) output into separate file
 - `--kernel-name kns=some_substring`
 - Only checks kernels containing "some_substring"

compute-sanitizer

Example

- Compile code with debug information
- Run **memcheck**:

```
srun -n 4 compute-sanitizer \  
--log-file jacobi.%q{SLURM_PROCID}.log \  
--save jacobi.%q{SLURM_PROCID}.compute-sanitizer \  
./jacobi -niter 10
```
- Actual output can be very long, if many GPU threads produce (similar) errors
- Stores one *.log file per MPI rank:
 - %q{ENV_VAR} supported by all the NVIDIA tools discussed here, embed environment variable in file name
 - ENV_VAR should be one set by the process launcher, and provide a unique ID
 - Evaluated only once tool starts running (on compute node) – not when launching job

OpenMPI :
OMPI_COMM_WORLD_RANK
OMPI_COMM_WORLD_LOCAL_RANK

MVAPICH2 :
MV2_COMM_WORLD_RANK
MV2_COMM_WORLD_LOCAL_RANK

Slurm:
SLURM_PROCID
SLURM_LOCALID

<https://www.open-mpi.org/faq/?category=running#mpi-environmental-variables>
<http://mvapich.cse.ohio-state.edu/static/media/mvapich/mvapich2-userguide.html#x1-34600013>
https://slurm.schedmd.com/srun.html#SECTION_OUTPUT-ENVIRONMENT-VARIABLES

compute-sanitizer

Example

- Abbreviated output:

```
===== COMPUTE-SANITIZER
[....]
===== Invalid __global__ write of size 4 bytes
=====      at 0x6d0 in mpi/jacobi_kernels.cu:60:initialize_boundaries(float*, float*,
float,                                     int, int, int,
int)
=====      by thread (1,0,0) in block (32,0,0)
=====      Address 0x14fb88020000 is out of bounds
=====      Saved host backtrace up to driver entry point at kernel launch time
=====      Host Frame: [0x20d6ea]
=====      in libcuda.so.1
=====      Host Frame: [0x115ab]
[....]
=====
===== ERROR SUMMARY: 10 errors
```

- We introduced an off-by-one error in line 60:

```
a_new[iy * nx + (nx - 1) + 1] = y0;
```




DEBUGGING

Before resorting to a debugger

...write debuggable software

- Do **error handling**
 - Use CUDA's error handling API https://docs.nvidia.com/cuda/cuda-runtime-api/group__CUDART__ERROR.html
 - Distinguish between **debug and release builds** and test both iteratively during development
 - Write representative **test cases**
 - Consider most relevant scientific use cases, walk relevant code paths
 - Should run quickly, and with limited compute resources
 - Some (but not all) tests at full scale
 - Think about **maintainability** and **extensibility** of your code
 - Which parts must really depend on each other?
 - Where to use libraries?
 - Today's code is tomorrow's legacy
- ➡ Refactor as you go while avoiding full rewrites - code encapsulates hard-earned bug fixes and knowledge!

Before resorting to a debugger

...make debugging as simple as possible

- Determine how to reproduce the error:
 - Occurs for all inputs or only for specific ones?
 - Occurs in every run or only occasionally?
- Vary kernel configuration to find the simplest scenario that reproduces the error:
 - Does the bug occur under sequential execution?
 - Does it occur when executed on a single thread block?
 - Does it occur when executed on multiple thread blocks?
- Identify concerned code section/kernel function

➡ Determine how to reproduce the bug, and debug in the simplest environment possible

Before resorting to a debugger

...consider the type of bug you observe

- Debugging approach depends on the characteristics of the bug you observe:
 - Crash  stacktrace often points to location of bug
 - Wrong result, deterministically  analyze compute logic, e.g. interactively with cuda-gdb
 - Wrong result, non-deterministically  check for race conditions, e.g. via cuda-gdb and racecheck (for shared memory data races only) and synccheck
 - Hang  check for deadlock/livelock, e.g. via cuda-gdb and racecheck and synccheck

 Observe program behavior and let it guide your approach

Before resorting to a debugger

... use static and dynamic checking techniques to catch bugs early

- Use **static checking techniques**
 - Enable compiler warnings e.g. `-Wall -Wextra`
- Use **dynamic checking techniques** during development
 - Add assertions to document implicit assumptions and catch bugs early
 - Compile with debug information to receive meaningful stacktraces on crash, and for usage with debugger
 - Check for memory corruption (leaks, out-of-bounds, etc.) precautionary and automatically e.g. with tools such as *Valgrind* and *Compute Sanitizer's memcheck*

NVCC compiler flags to generate debug information

<code>-g</code>	Embed symbol info for <i>host</i> code
<code>-G</code>	Generate debug information for <i>device</i> code → reduces performance
<code>-lineinfo</code>	Generate line correlation info for <i>device</i> code → useful to debug optimized code

cuda-gdb

Extends *gdb* for CUDA applications

- Interactive command-line debugger with extensions for GPU code
 - <https://docs.nvidia.com/cuda/cuda-gdb/index.html>
 - **gdb** commands are unchanged, and extensions for CUDA are prefixed with **cuda** keyword
 - *Symbolic debugger* – leverage debug symbols to correlate runtime errors with original source code
 - Basic **workflow**:
 - Compile program with debug information
 - **nvcc -g -G prog.cu -o prog**
 - Call crashing program as such
 - **cuda-gdb --args ./prog arg1 arg2**
 - Opens debugger shell, showing prompt (cuda-gdb)
 - Launch program with **run**
- ➡ Step through execution to understand program flow, inspect and modify variables, observe execution contexts...
... and identify the cause of the bug! 😊

cuda-gdb example

GPU-specific commands

(cuda-gdb) **info cuda threads**

	BlockIdx	ThreadId	To	BlockIdx	ThreadId	Count	Virtual PC	Filename	Line
Kernel 0									
*	(0,0,0)	(0,0,0)		(0,0,0)	(31,0,0)	32	0x0000000000acbf90	gpu_print.cu	19
	(1,0,0)	(0,0,0)		(1,0,0)	(31,0,0)	32	0x0000000000acbf60	gpu_print.cu	18

Get state of threads, and current focus (*)

Current focus of debugger

(cuda-gdb) **cuda thread**

thread (0,0,0)

Change focus

(cuda-gdb) **cuda thread 10**

[Switching **focus** to CUDA kernel 0, grid 1, block (0,0,0), **thread (10,0,0)**, device 0, sm 0, warp 0, lane 10]

19 printf("blockIdx.x = %d, threadIdx.x = %d, i = %d\n", blockIdx.x, threadIdx.x, i);

- Focus can be set to specific blocks, SMs, devices, ...
 - Hardware and software abstractions e.g. blocks vs. SMs



try: (cuda-gdb) **help cuda**

- Options



try: (cuda-gdb) **set cuda<ENTER>**

- Two commonly-used options: **api_failures** and **launch_blocking**

cuda-gdb

Cheat sheet for CUDA specific commands

- Navigating execution context

<code>info cuda devices</code>	List active GPUs and their properties
<code>info cuda kernels / blocks / threads</code>	Display state of active kernels / blocks/ threads
<code>cuda thread</code>	Display current focus
<code>cuda block <bid> thread <tid></code>	Switch focus to thread <tid> in block <bid>

- Settings that can facilitate debugging

<code>set cuda break_on_launch application</code>	Set breakpoints at all application kernel launches
<code>set cuda launch_blocking on</code>	Force synchronous kernel launches
<code>set cuda api_failures stop</code>	Quick way to check CUDA API failures e.g. launch errors

Approaches for Multi-Process Applications

- Tools usually run on a single process – adapt for highly distributed applications?
 - Bugs in parallel programs are often serial bugs in disguise
- Common MPI theme: workload is distributed; correctness/performance bug similar for all processes
- Run tool N times in parallel, have N output files, only look at 1 (or 2, ...)
- Even works for load imbalance (profile the rank spending least amount of time waiting in barriers)
- For parallel race conditions parallel tools are best
- How to get unique tool output?
 - `%q{ENV_VAR}` supported by all the NVIDIA tools discussed here, embed environment variable in file name
 - Other tools: use a launcher script, for late evaluation

Using cuda-gdb with MPI

Programmatically wait for debugger to attach

- Launcher (mpirun/srun/...) complicates starting process inside debugger
- Workaround: attach later

```
#include <unistd.h>
if (rank == 0) {
    char name[255]; gethostname(name, sizeof(name)); bool attached;
    printf("rank %d: pid %d on %s ready to attach\n", rank, getpid(), name);
    while (!attached) { sleep(5); }
}
```

- Launch process, sleep on particular rank

```
$ srun -n 4 ./jacobi -niter 10
rank 0: pid 28920 on jwb0001.juwels ready to attach
```

- Then attach from another terminal (flags are system dependent)

```
[jwlogin]$ srun -n 1 --overlap -w jwb0001.juwels --jobid <jobid> --pty bash -i
[jwb0001]$ cuda-gdb --attach 28920
```

- Wake up sleeping process and continue debugging normally
(cuda-gdb) set var attached=true

JSC system shortcut:
sgoto <jobid> <node>

Using cuda-gdb with MPI

Environment variables for easier debugging

- Automatically wait for attach on exception without code changes:

```
$ CUDA_DEVICE_WAITS_ON_EXCEPTION=1 srun ./jacobi -niter 10
Single GPU jacobi relaxation: 10 iterations on 16384 x 16384 mesh with norm check every 1
iterations
jwb0129.juwels: The application encountered a device error and CUDA_DEVICE_WAITS_ON_EXCEPTION is
set. You can now attach a debugger to the application (PID 31562) for inspection.
```

- Same as before, go to node (see previous slide), then attach cuda-gdb:

```
$ cuda-gdb --pid 31562
CUDA Exception: Warp Illegal Address
The exception was triggered at PC 0x508ca70 (jacobi_kernels.cu:88)
```

```
Thread 1 "jacobi" received signal CUDA_EXCEPTION_14, Warp Illegal Address.
[Switching focus to CUDA kernel 0, grid 4, block (0,0,0), thread (0,20,0), device 0, sm 0, warp 21,
lane 0]
0x00000000508ca80 in jacobi_kernel<32, 32><<<(512,512,1),(32,32,1)>>> (/*...*/) at jacobi_kernels.c
u:88
88          real foo = *((real*)nullptr);
```


Debugging MPI+CUDA applications

More environment variables for offline debugging

- With `CUDA_ENABLE_COREDUMP_ON_EXCEPTION=1` core dumps are generated in case of an exception
 - `CUDA_ENABLE_LIGHTWEIGHT_COREDUMP=1` does not dump application memory - faster
 - Can be used for post-mortem debugging
 - Helpful if live debugging is not possible
- Enable/Disable CPU part of core dump (enabled by default)
 - `CUDA_ENABLE_CPU_COREDUMP_ON_EXCEPTION`
- Specify name of core dump file with `CUDA_COREDUMP_FILE`
- Open GPU core dump
 - (cuda-gdb) target `cuda` core.cuda
- Open CPU+GPU core dump
 - (cuda-gdb) target `core` core.cpu core.cuda

<https://docs.nvidia.com/cuda/cuda-gdb/index.html#gpu-core-dump-support>

Example: Opening a Core Dump

- Running and generating the core file

```
$ CUDA_ENABLE_COREDUMP_ON_EXCEPTION=1 CUDA_ENABLE_LIGHTWEIGHT_COREDUMP=1 srun ./jacobi -niter 10
Single GPU jacobi relaxation: 10 iterations on 16384 x 16384 mesh with norm check every 1 iterations
srun: error: jwb0021: tasks 0-3: Aborted (core dumped)

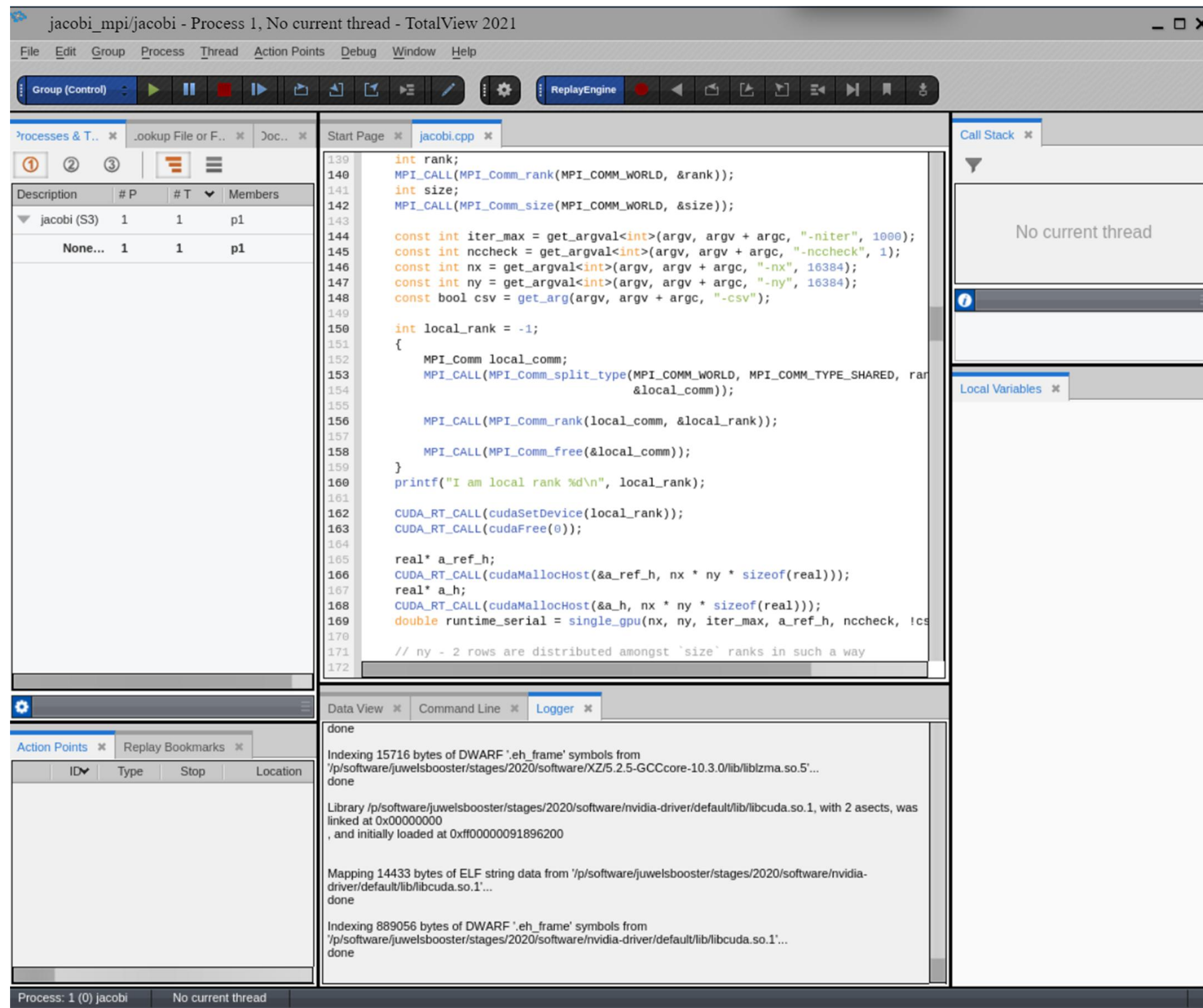
$ ls core*
core.jwb0021.juwels.23959  core_1633801834_jwb0021.juwels_23959.nvcudmp ...
```

- And opening the core dump in `cuda-gdb`

```
(cuda-gdb) target cudacore core_1633801834_jwb0021.juwels_23959.nvcudmp
Opening GPU coredump: core_1633801834_jwb0021.juwels_23959.nvcudmp
[New Thread 23979]

warning: No exception was found on the device
[Current focus set to CUDA kernel 0, grid 4, block (0,0,0), thread (0,2,0), device 0, sm 0, warp 0, lane 0]
#0  0x0000000057e1ae0 in void jacobi_kernel<32, 32>(float*, float const*, float*, int, int, int, bool)
    <<<(512,512,1),(32,32,1)>>> ()
    at /p/project/cexalab/hrywniak1/code/multi-gpu-programming-models/mpi/jacobi_kernels.cu:87
87          real foo = *((real*)nullptr);
(cuda-gdb)
```


Specialized parallel debuggers



- `cuda-gdb` can debug multiple processes (`add-inferior`), although...
- For truly parallel bugs (e.g. multi-node, multi-process race conditions), third-party tools offer more convenience
 - Or enable „live“ analysis in the first place
- Linaro DDT (formerly ARM)
- Perforce TotalView (screenshot)
 - Available as module on JSC systems



PROFILING

Debugging Performance

Why you **must** use profilers

- Profilers are **measurement tools** that help to **optimize performance** *efficiently*
- Paraphrasing Donald Knuth:
 - Don't overoptimize, but meta-optimize your own time by **using tools to focus on relevant parts**
- Do not trust your gut instinct – very often very misleading
 - Easy to waste a lot of time chasing the "perceived" issue
- Getting the same information, you might end up reimplementing your own profiler
- Profilers for different scenarios
 - Multi-process vs. single-process vs. kernel-level profiling
- Different performance collection methods, different tradeoffs
 - Sampling
 - Tracing
 - Instrumentation

Performance optimization is an iterative workflow!

4. Deploy and Test

1. Assess

- Profile
- Identify hotspots
- Identify performance limiters

3. Optimize

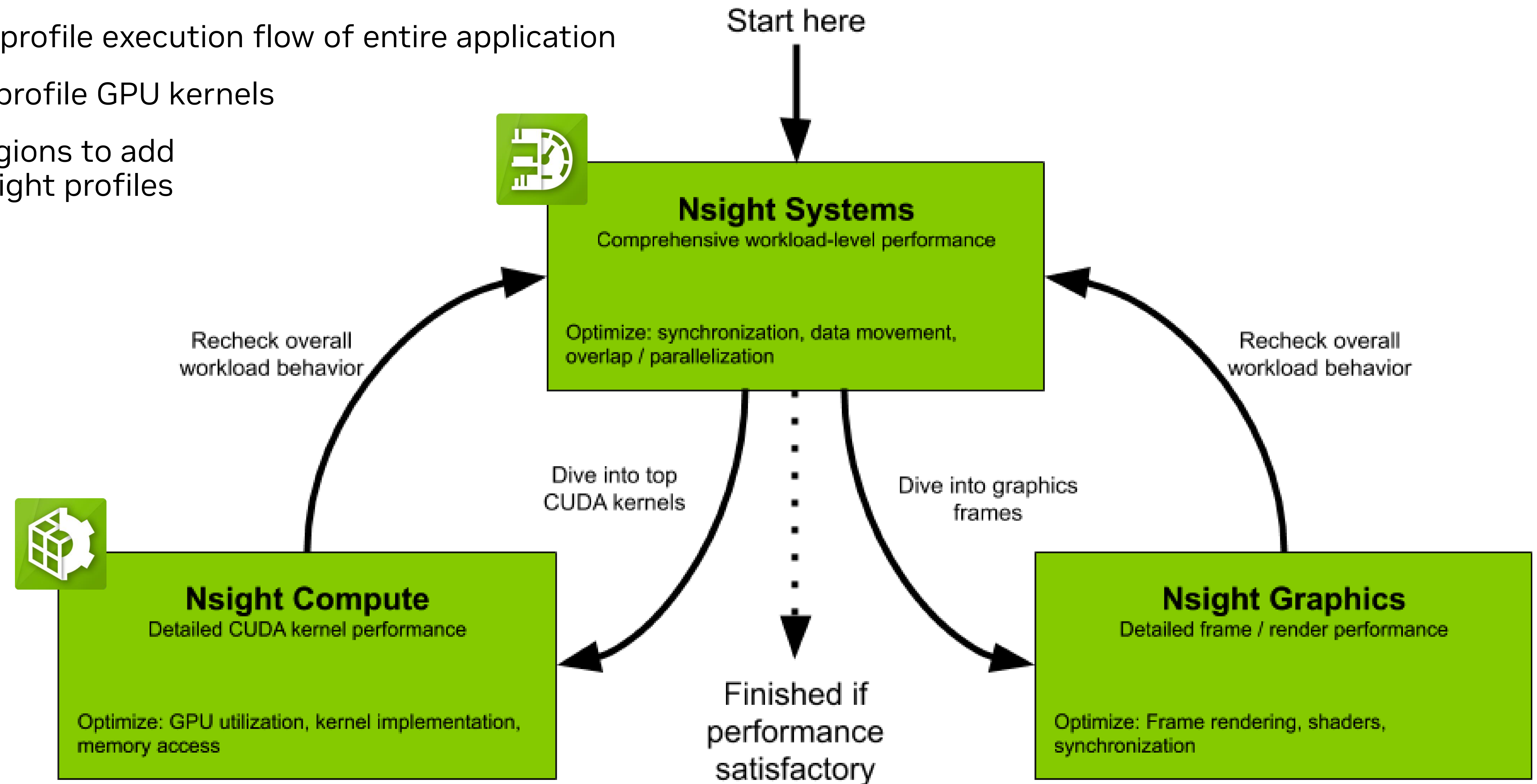
2. Parallelize



The Nsight Suite Components

How the pieces fit together

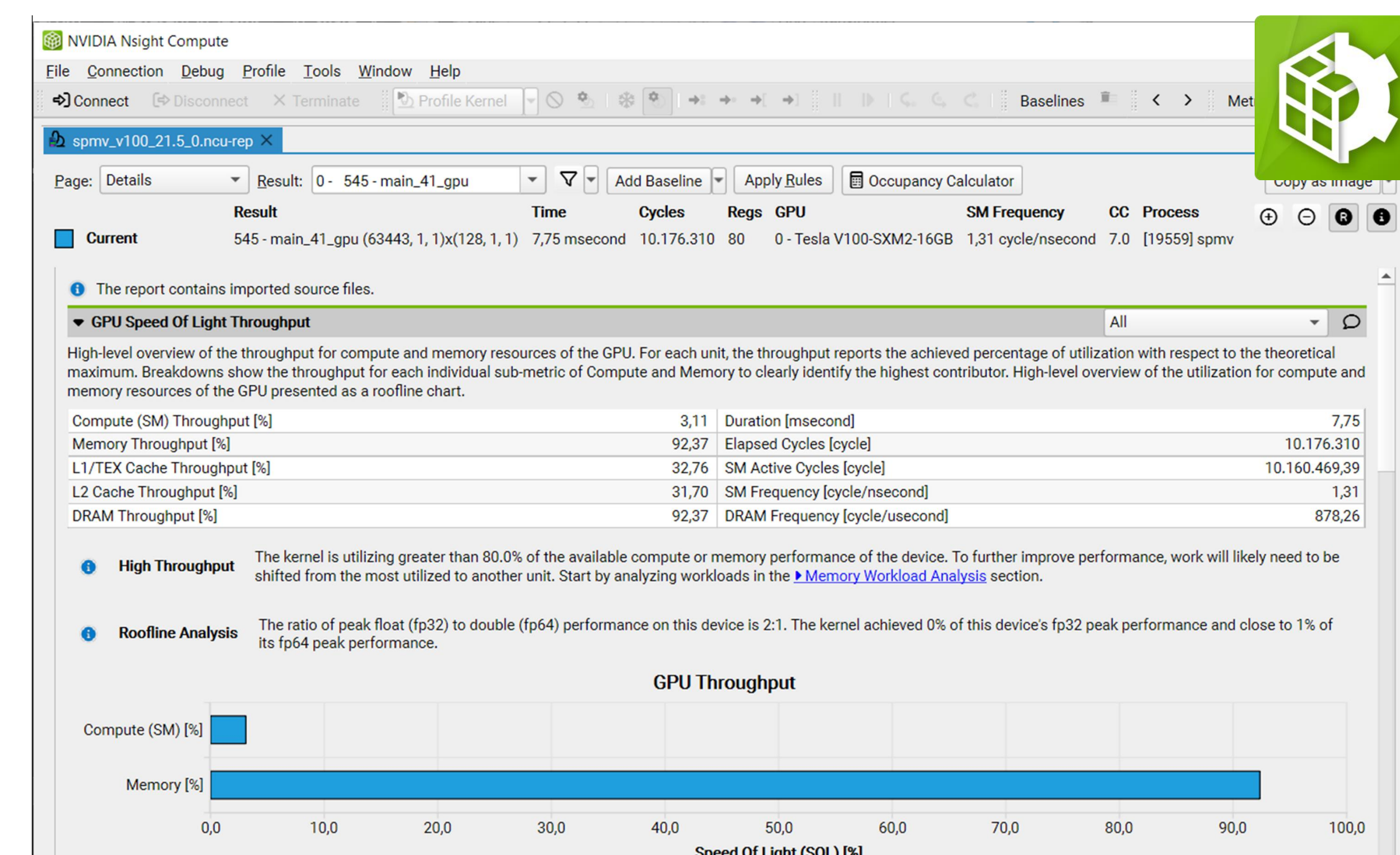
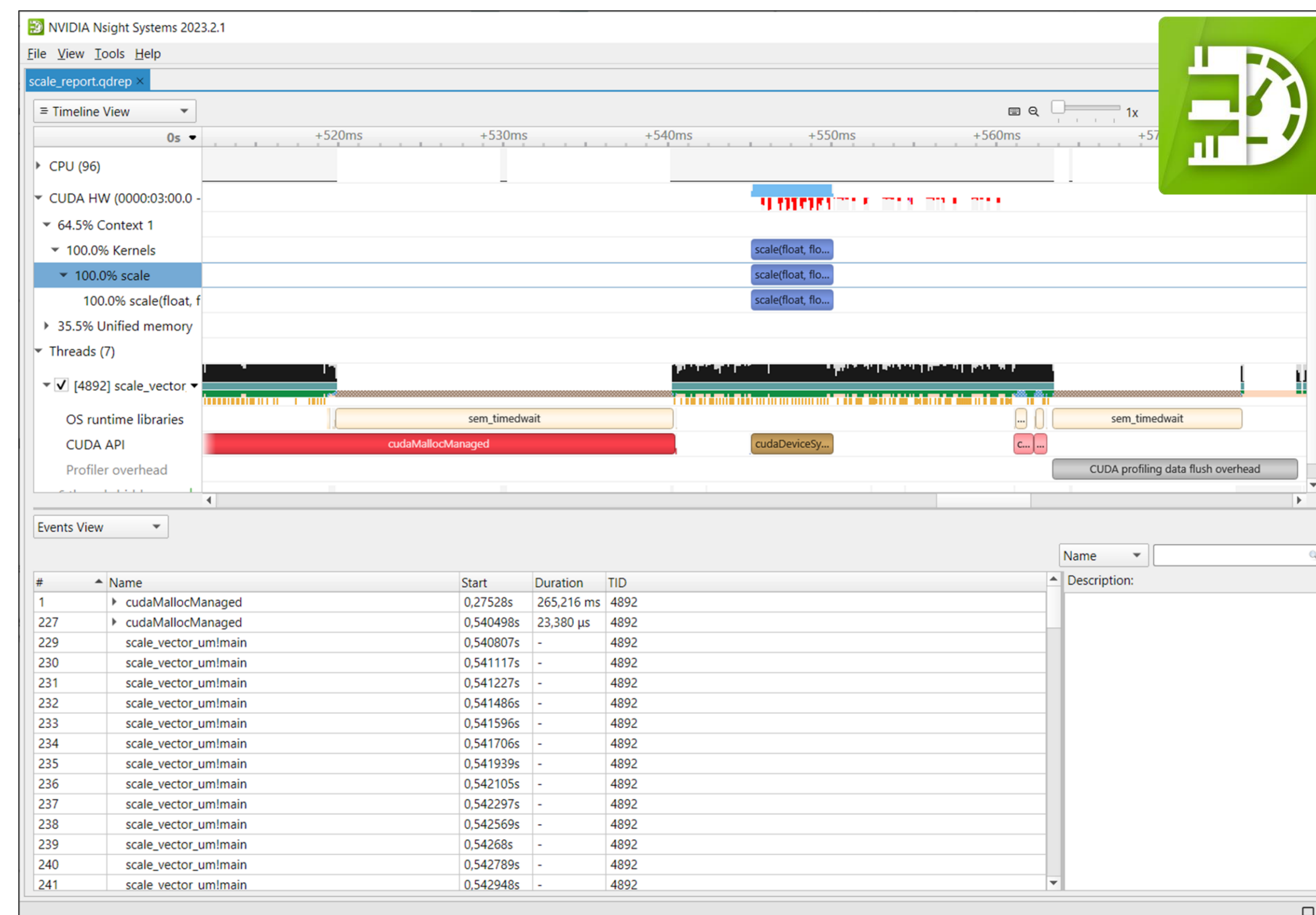
1. Use **Nsight Systems** **nsys** to profile execution flow of entire application
2. Use **Nsight Compute** **ncu** to profile GPU kernels
3. Annotate code with **NVTX** regions to add contextual information to Nsight profiles



Where Should I Start Profiling?

And which tool to use?

- Always tradeoff between slightly conflicting goals: Performance vs. Maintainability vs. Effort
- Start with profiling at system-level view → **Nsight Systems**
- Ensure you understand your timeline, and where the GPU is active/inactive
 - What happens only once, what every time step?
 - How does the timeline change for different workloads?
- Take the low-hanging fruits first!
- Don't shy away from kernel optimization but consider effort vs. impact on overall application performance
 - Consider Amdahl's Law: Assume optimized kernel takes 0 s, how large is speedup of entire application?
 - If whole timeline consists of a single kernel, by all means, start profiling and optimizing it → **Nsight Compute**



Interlude - Maximum achievable speedup

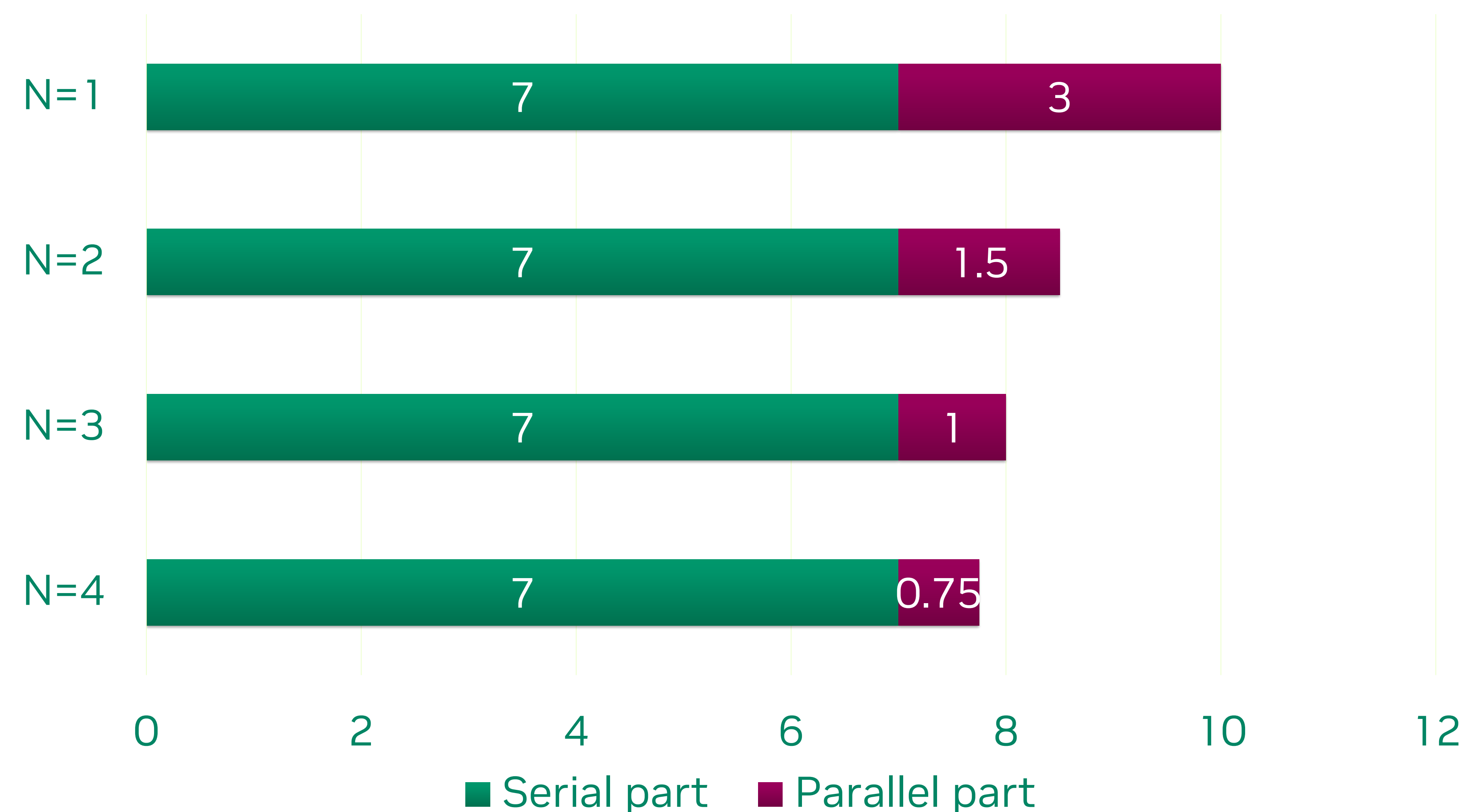
Amdahl's law

- Amdahl's law states overall speedup s given the parallel fraction p of code and number of processes N

$$s = \frac{1}{1 - p + \frac{p}{N}} < \frac{1}{1 - p}$$

- Limited by serial fraction, even for $N \rightarrow \infty$
- Example for $p = 30\%$
- Generally applicable on any level,
e.g. also valid for per-method speedups

Using 1 to 4 processes, total runtime



A first (I)Nsight

Recording and inspecting Nsight Systems profiles

- Record profile using the command line
`nsys profile --trace=cuda,nvtx,mpi --output=my_report.%q{SLURM_PROCID} ./app`
- ...and inspect profiling results by opening report in `nsys-ui` via X-forwarding or locally
- Record profile using the command line, adding `--stats true`
`nsys profile --stats true --trace=cuda,nvtx,mpi --output=my_report.%q{SLURM_PROCID} ./app`
- ...and run reports on command line to check validity of profile, and identify important kernels

Several customizable reports based on sqlite + Python included in nsys installation

[4/6] Executing 'cuda_gpu_kern_sum' stats report

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
70.3	6,305,096	1,001	6,298.8	4,768.0	4,704	1,544,154	48,655.6	jacobi_iteration(float *, const float *, int, int, float)
29.7	2,663,124	1,001	2,660.5	2,592.0	2,496	3,200	130.1	apply_periodic_bc(float *, int, int)

Nsight Systems GUI

Main timeline view, Events View

Use zoom (+/-) to inspect closely

Find log files, execution command, execution environment and more meta-data of the profile

CPU utilization

CUDA kernel occupancy

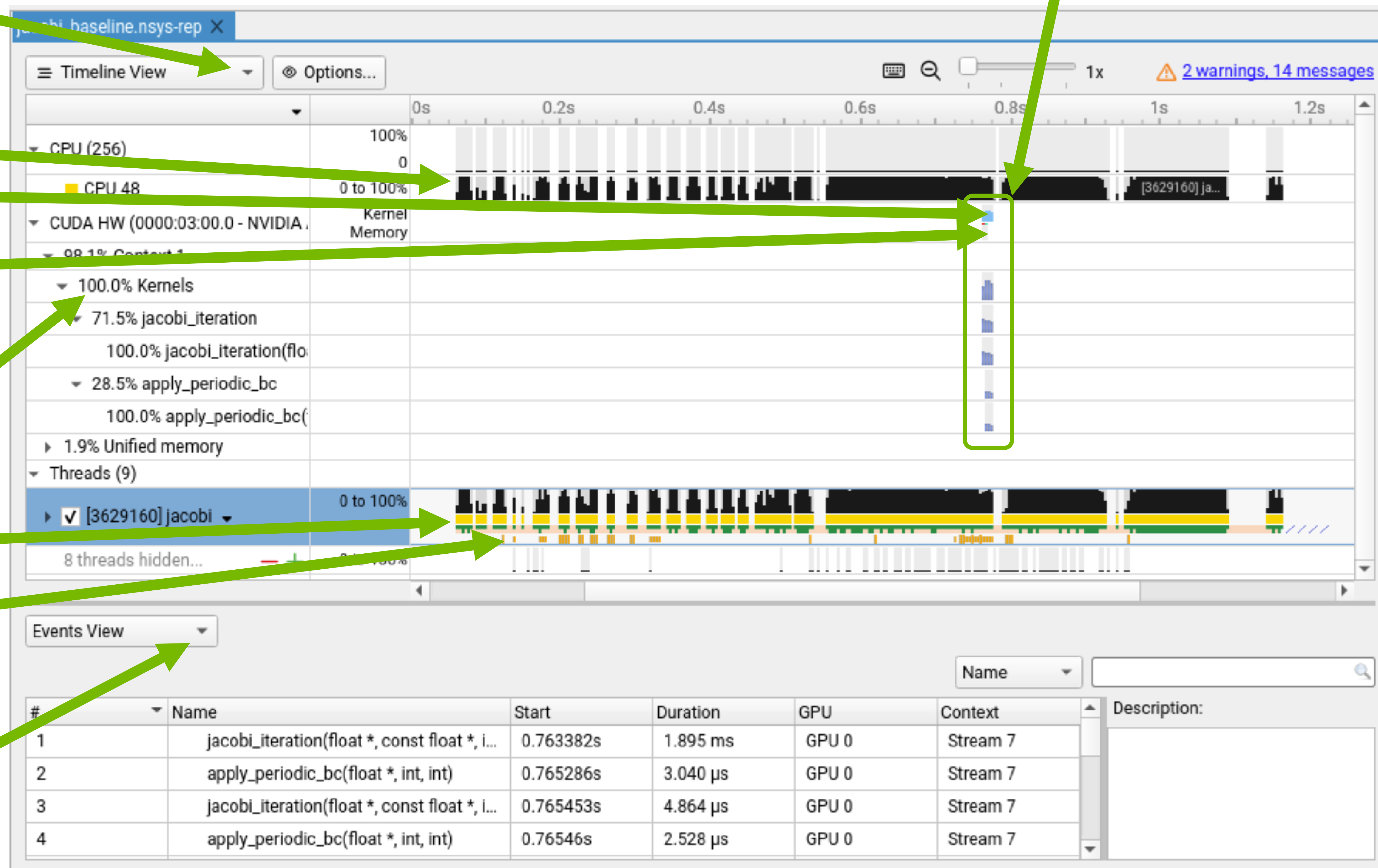
CUDA memory transfers

Right-click to show in Events View

Current core and state of CPU thread

Sampling points incl. stack traces

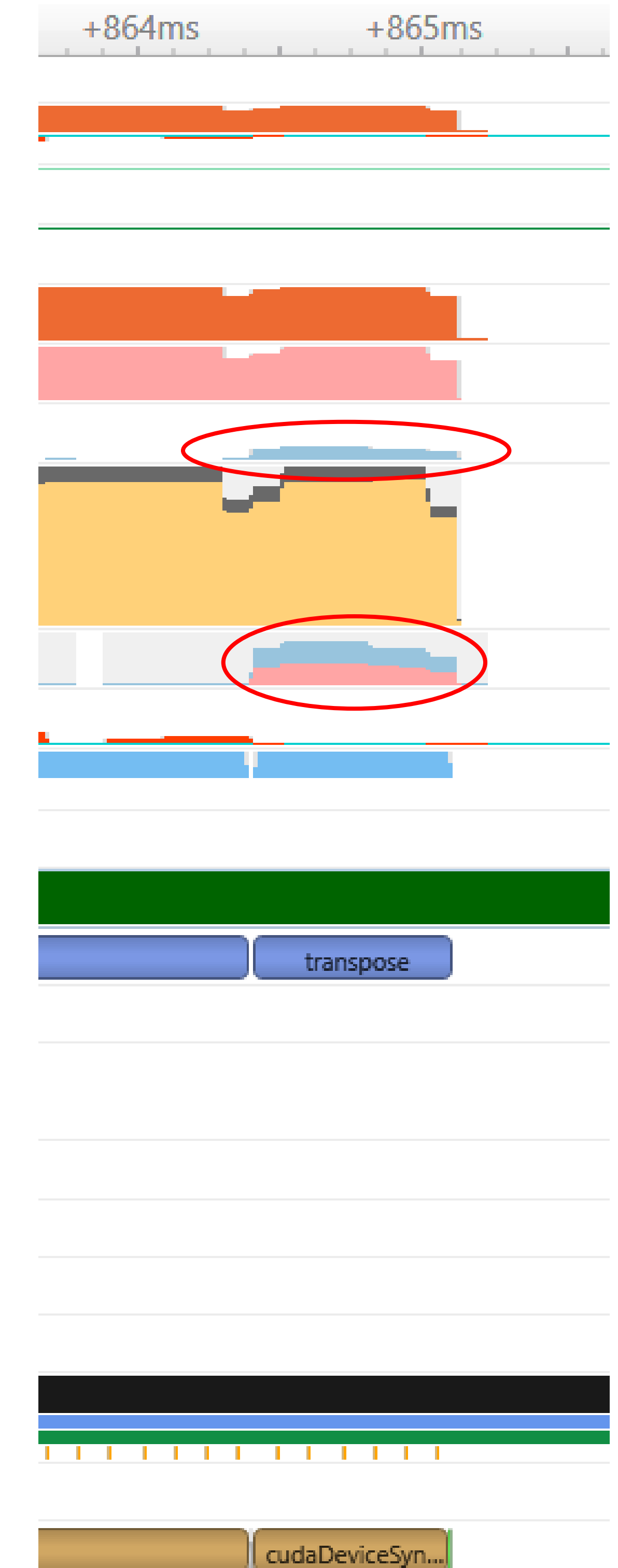
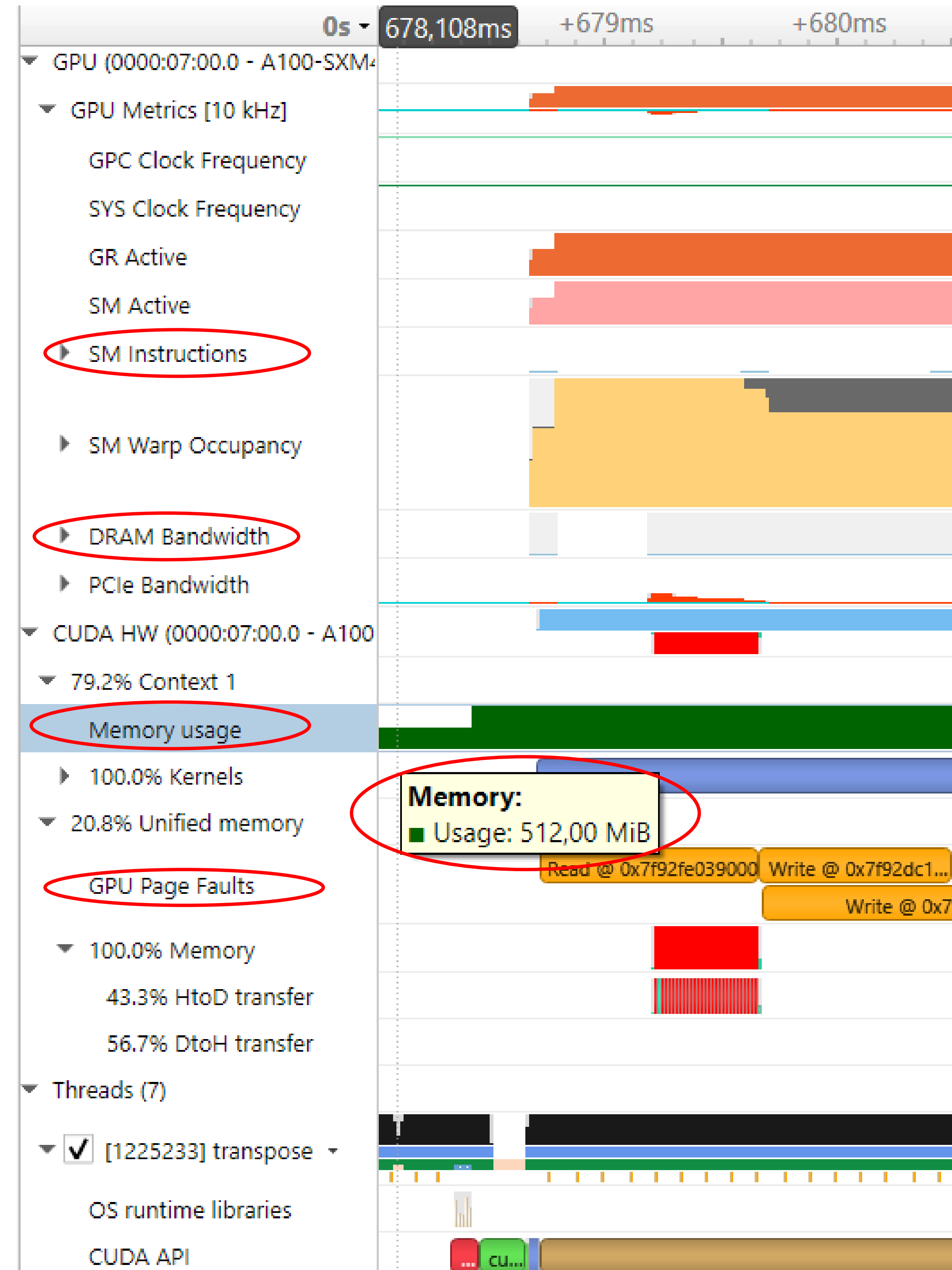
Events views, stats system view, expert system view



GPU Metrics in Nsight Systems

...can save some kernel profiling effort

- Valuable low-overhead insight into HW usage
 - SM instructions
 - DRAM Bandwidth
 - PCIe Bandwidth (RDMA)
 - Analyze memory usage
 - Detect unified memory page faults
 - ...
- nsys profile
 - gpu-metrics-devices=0
 - cuda-memory-usage=true
 - cuda-um-cpu-page-faults=true
 - cuda-um-gpu-page-faults=true
 - ./app



NVIDIA Tools Extension Library

Range-based code annotation API to provide context to tools

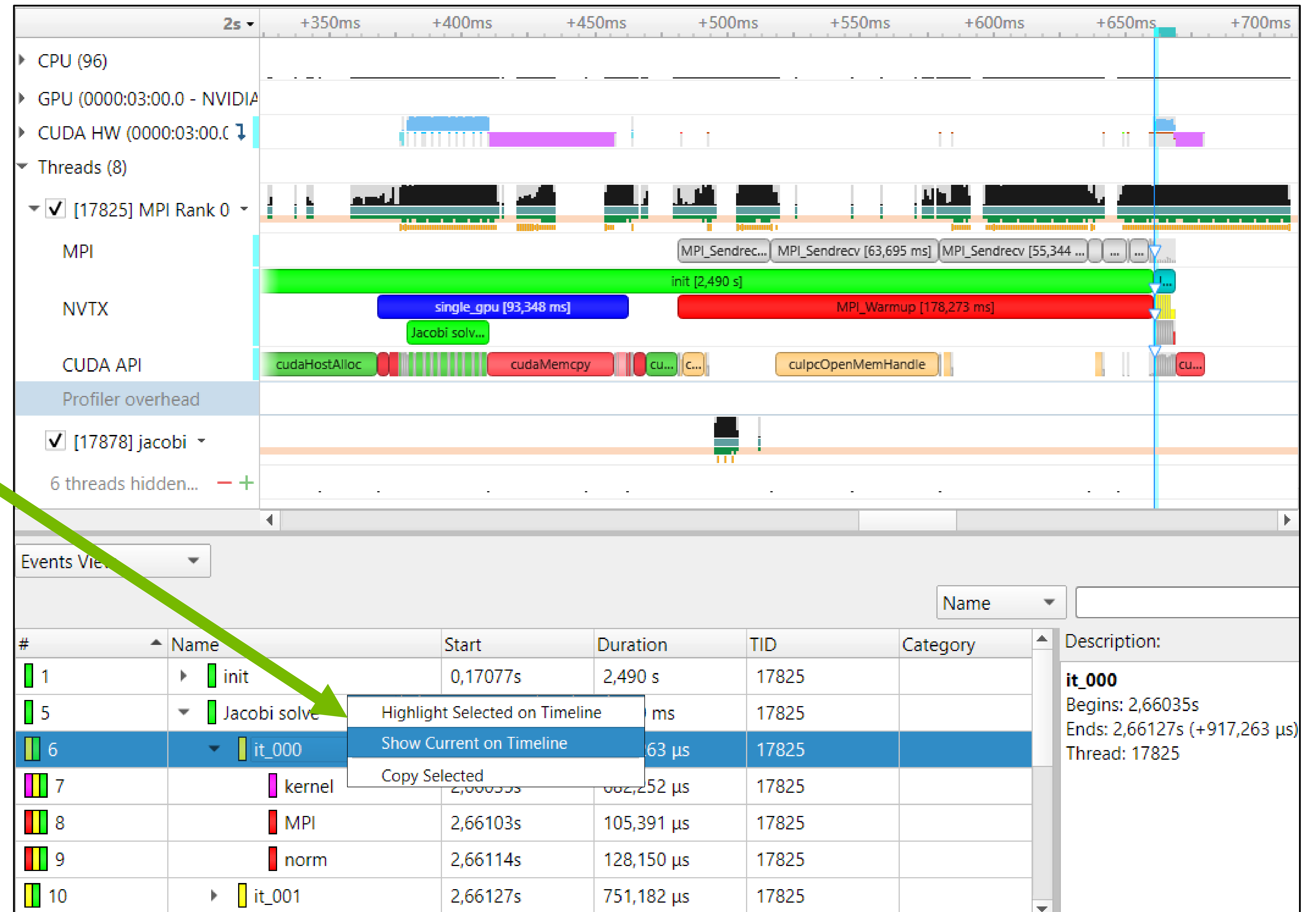
- `#include "nvtx3/nvToolsExt.h"`
 - NVTX v3 is header-only
 - C++ and Python APIs
- Fortran: [NVHPC compilers include module](#)
 - Just use `nvtx` and `-lnvhpcwrapnvtx`
 - Other compilers: See blog post linked below
- Use PUSH/POP macros to define ranges conveniently:
 - `PUSH_RANGE(name, color_idx)`
→ <https://developer.nvidia.com/blog/cuda-pro-tip-generate-custom-application-profile-timelines-nvtx/>
- Sprinkle them strategically through code
 - → Use hierarchically: nest ranges

```
int main(int argc, char** argv) {
    PUSH_RANGE("main", 0)
    PUSH_RANGE("init", 1)
    do_initialization();
    POP_RANGE
    /* ... */
    PUSH_RANGE("computation", 2)
    jacobi_kernel<<< /* ... */, compute_stream>>>(...);
    cudaStreamSynchronize(compute_stream);
    POP_RANGE
    /* ... */
    POP_RANGE
}
```


NVIDIA Tools Extension Library

...adding context to the profile

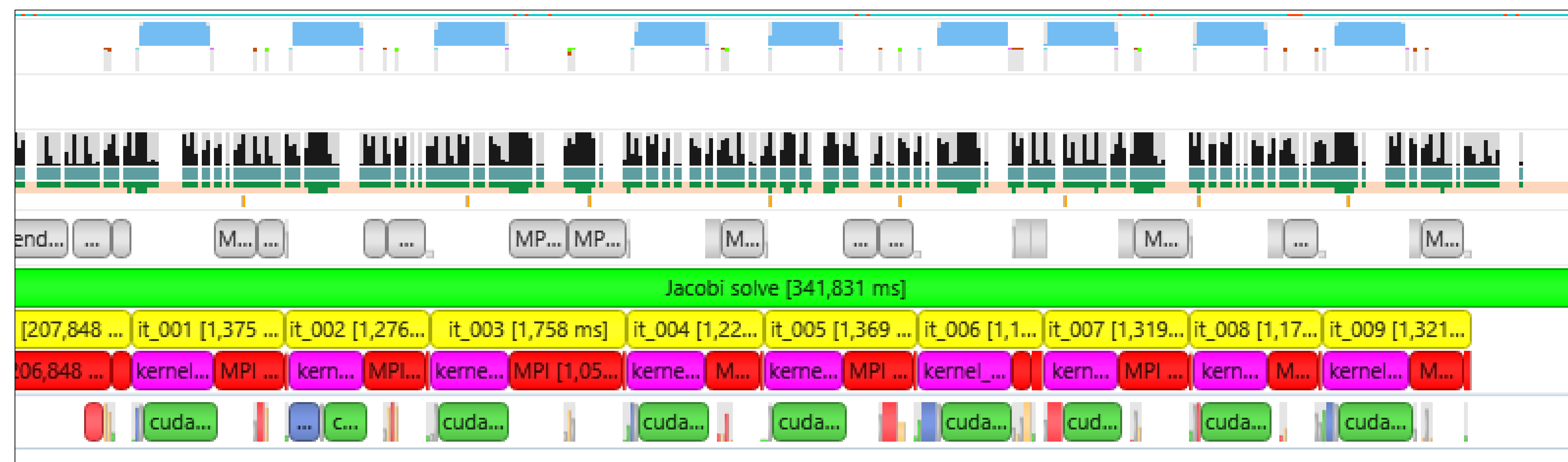
- Like manual timing but less work
- Allows nesting
- Useful for correlation and filtering
- Use events view for quick navigation



Minimizing Profile Size

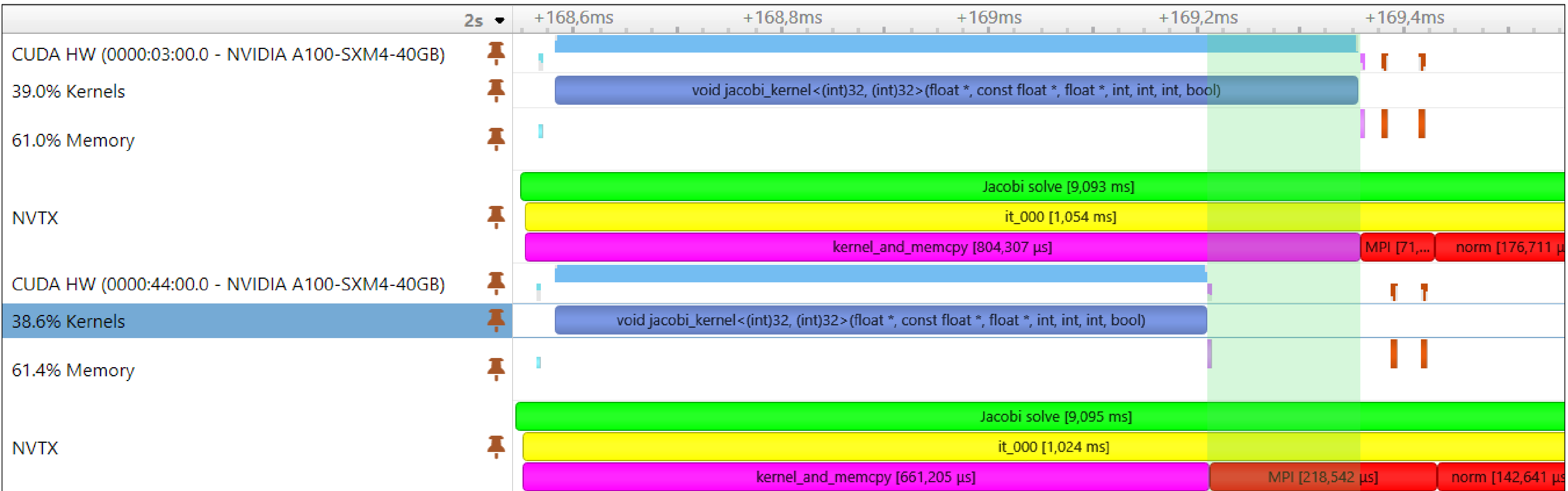
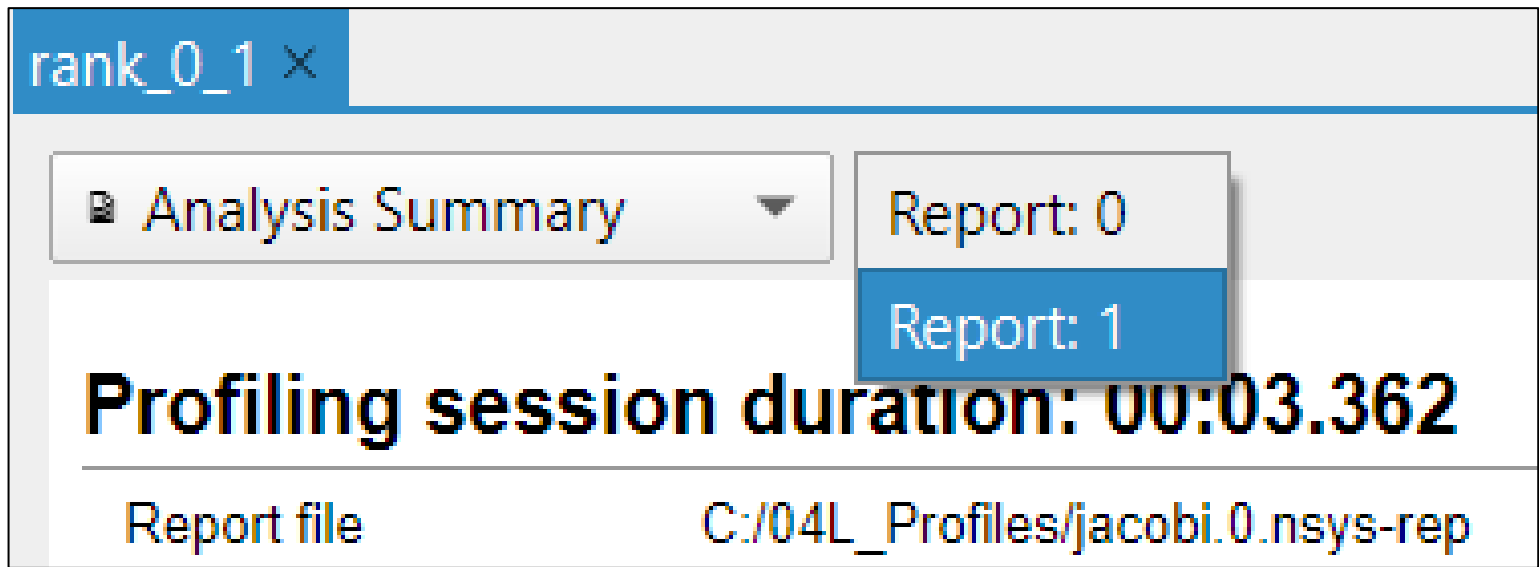
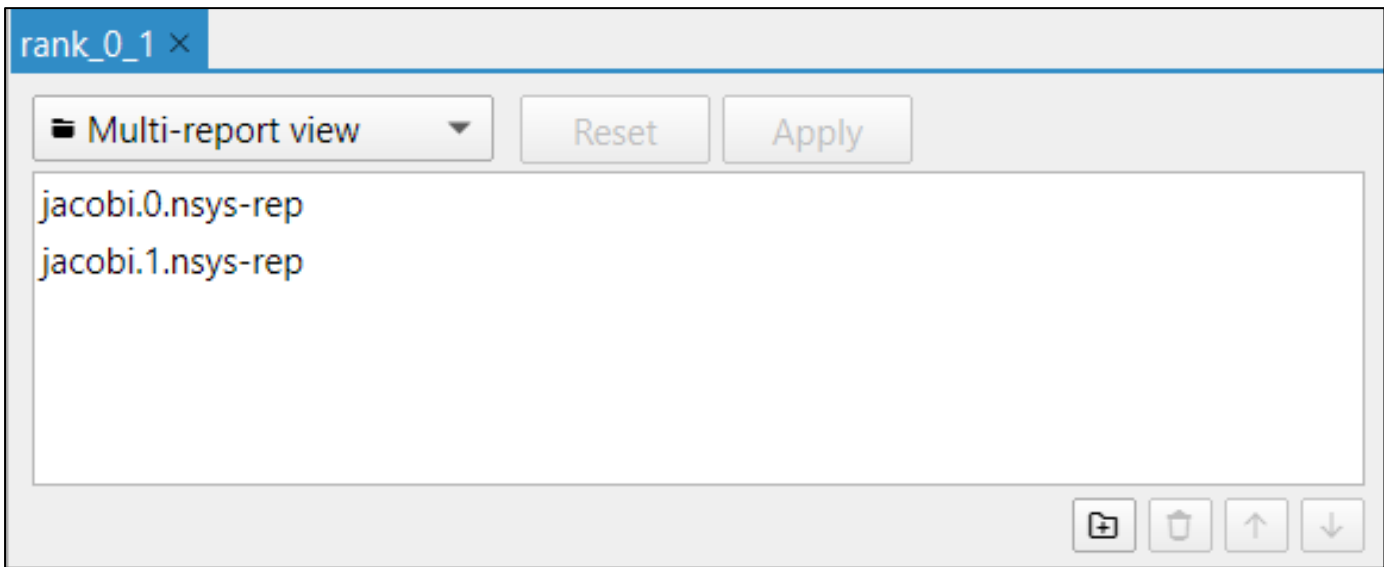
Shorter time, smaller files = quicker progress

- Only profile what you need – all profilers have some overhead
 - Example: Event that occurs after long-running setup phase
- Bonus: lower number of events leads to smaller file size
- Add to nsys command line:
 - `--capture-range=nvtx --nvtx-capture=any_nvtx_marker_name \`
`--env-var=NSYS_NVTX_PROFILER_REGISTER_ONLY=0 --kill none`
 - Use [NVTX registered strings](#) for best performance
- Alternatively: `cudaProfilerStart()` and `-Stop()`
 - `--capture-range=cudaProfilerApi`



Multi-Process GPU Analysis

- Load multiple reports into timeline
 - analyze differences in execution, GPU utilization
- Pin rows for comparison
- Example: End time of kernel execution

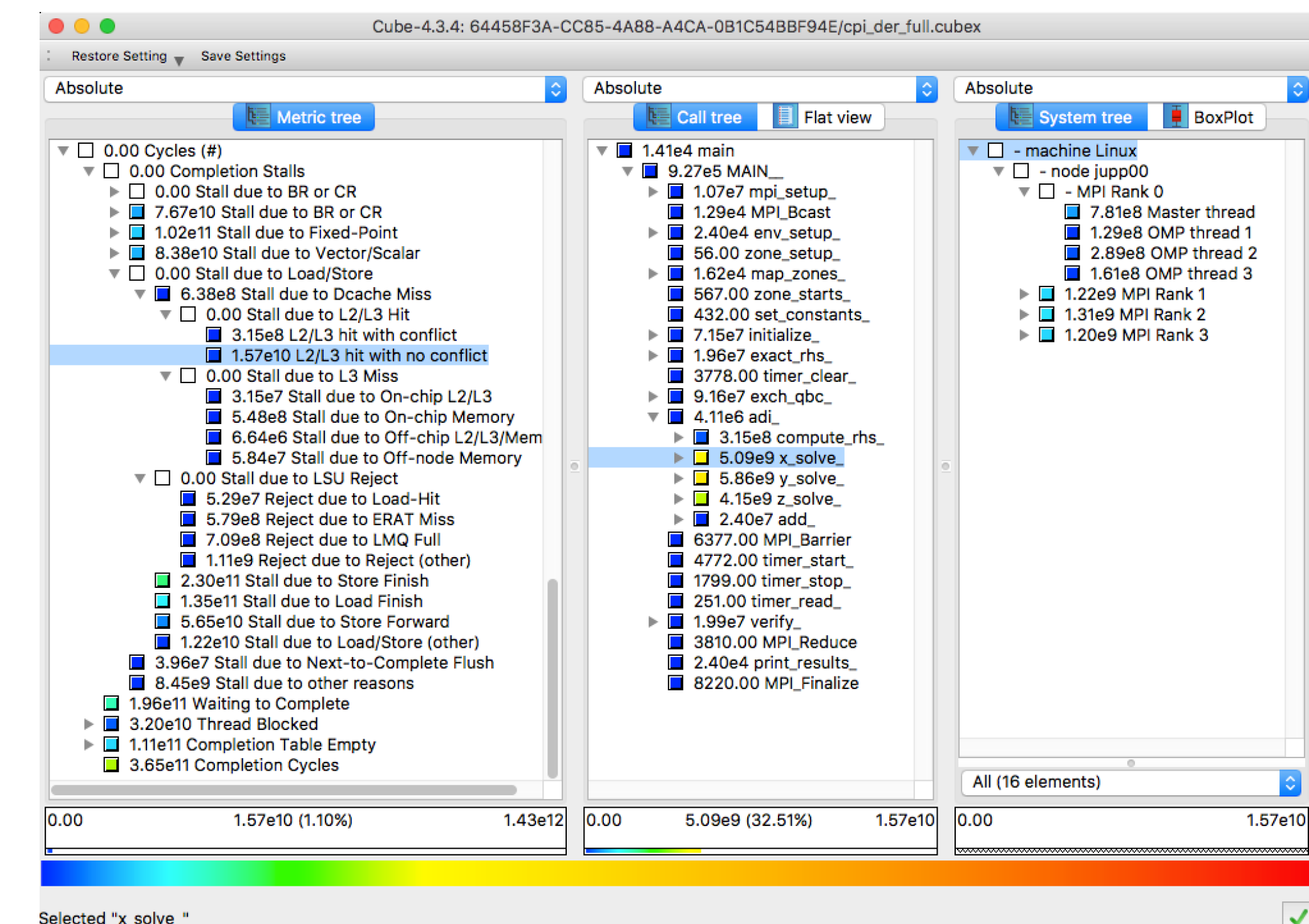


Community Profiling Tools

Specialized for large-scale distributed analysis



- Detecting issues at scale of thousands of GPUs (and processes)
 - Need to slice and dice data, too much to make sense of raw data
- Common measurement/instrumentation infrastructure: *Score-P*
 - Prefix all compilation/linker commands with `scorep -cuda`
- GPU performance counters available via CUPTI
→ CUDA Profiling Tools Interface
 - Build your own profiler (integration): <https://docs.nvidia.com/cupti/index.html>
- Score-P: Measurement infrastructure, can record CPU/GPU activities
 - Cube: display hierarchical info collected via Score-P
 - Vampir: analyze application traces, discover MPI issues
 - ... and many more



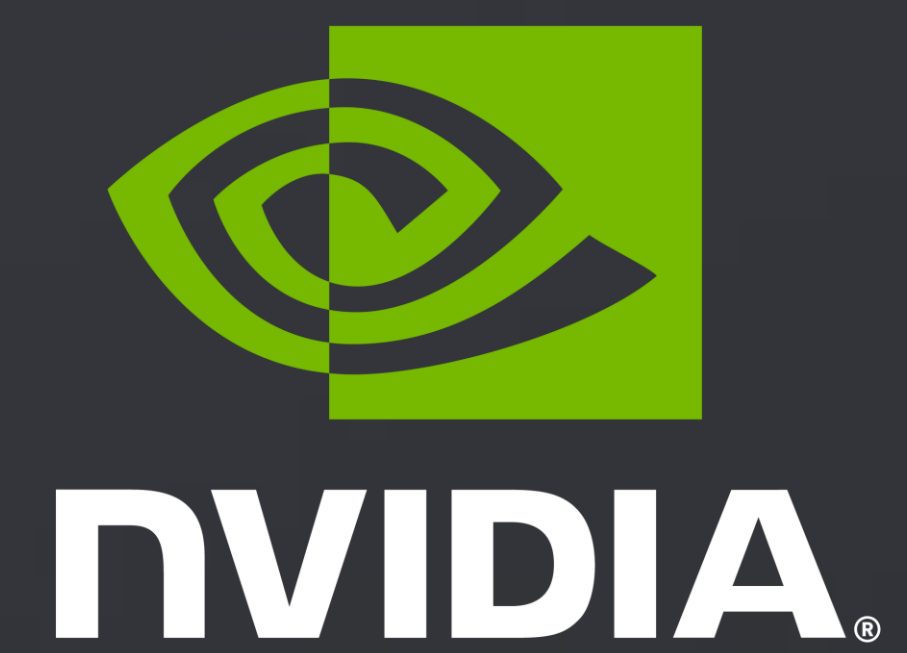
Summary

Correctness checking, debugging, profiling

- Overview of GPU developer tools
 - Correctness checking with compute-sanitizer tool suite
 - Debugging with cuda-gdb
 - System-level application profiling with Nsight Systems
- Profiling is the foundation of well-informed performance optimization
- Performance optimization follows iterative workflow

Further Material

- GTC on-demand talks
 - [What, Where, and Why? Use CUDA Developer Tools to Detect, Locate, and Explain Bugs and Bottlenecks \(s41493, GTC 2022\)](#)
 - [Tuning GPU Network and Memory Usage in Apache Spark \(s31566, GTC 2022\)](#)
- Documentation for [cuda-gdb](#), [compute-sanitizer](#) and [Nsight Systems](#) and blog posts:
 - [Efficient CUDA Debugging: Memory Initialization and Thread Synchronization with NVIDIA Compute Sanitizer](#)
 - [Optimizing CUDA Memory Transfers with NVIDIA Nsight Systems](#)
 - ...and other posts: <https://developer.nvidia.com/blog/tag/nsight/>
- GTC labs from Nsight teams: <https://github.com/NVIDIA/nsight-training>
- GPU bootcamp material, e.g., https://github.com/openhackathons-org/nways_accelerated_programming



Q & A

